

Deadlock Detection and Replay of Multi-Thread Programs Based on Refined Segmentation and Lock Graphs

Faming Lu, Xiaoyu Wang, Guiyuan Yuan*, Jiajing Zheng, and Yunxia Bao

Abstract: Dynamic deadlock detection methods build segmentation graphs and lock graphs by extracting program behavior information from its running traces. However, existing segmentation graphs cannot model causal dependence caused by coupling of lock acquisition and thread fork operations. Lock sets in existing lock graphs cannot capture the information that a lock is held first and released soon. Both problems lead to false positives. In this work, segments in segmentation graphs are divided into fine-grained ones by identifying more causal dependence implied in a running trace. Segmentation graphs and lock graphs are extended with detailed lock acquisition and release information, based on which a deadlock detection method is proposed. To ensure authenticity of detected potential deadlocks, a deterministic scheduling scheme for deadlock replay is derived. In experiments, the proposed methods are implemented in CalFuzzer and its performance is better than PNULock and SegLock in terms of detection accuracy as well as time and memory overhead on eight Java multi-thread program examples.

Key words: dynamic deadlock detection; deadlock replay; program verification; lock graph

1 Introduction

Although parallel programming technology^[1, 2] improves program performance^[3, 4], software complexity and program scheduling uncertainty, e.g., deadlocks, data races, atomic violations, and use after free, are increasingly appearing^[5, 6]. Among them, deadlocks are the most representative ones, which may lead to system crashing and cause serious losses. Statistics show that about 30% of con-current defects

are related to deadlocks^[7]. Therefore, deadlock detection has become an urgent problem to improve software reliability and security.

Existing deadlock detection methods can be divided into three categories^[8], i.e., theorem proving and model checking^[9], data flow analysis^[10], and dynamic analysis^[11–14]. Theorem proving and model checking explore program execution paths and derive potential deadlocks by theory reasoning, which require manual participation. Data flow analysis introduces static analysis technologies to calculate static lock occupancy constraints or lock occupancy sequence graphs and uses constraint solving and cycle detection algorithms to detect potential deadlocks, which does not utilize program runtime information and leads to more false positives. Dynamic analysis extracts program behavior patterns by analyzing program running traces and then performs deadlock detection, which leads to fewer false positives and higher efficiency.

Considering the advantages of high efficiency of dynamic analysis, it has drawn a lot of attention.

• Faming Lu, Xiaoyu Wang, Guiyuan Yuan, and Jiajing Zheng are with College of Computer Science and Engineering, Shandong University of Science and Technology, Qingdao 266590, China. E-mail: fm_lu@163.com; 1370085436@qq.com; yuanguiyuan@sdust.edu.cn; 1793349953@qq.com.

• Yunxia Bao is with College of Mathematics and System Science, Shandong University of Science and Technology, Qingdao 266590, China. E-mail: baoyunxia98@163.com.

* To whom correspondence should be addressed.

Manuscript received: 2024-12-30; revised: 2025-03-22; accepted: 2025-04-18

Harrow Jr.^[15] firstly proposed a visual thread tool based on a lock graph and regards each cycle in it as a potential deadlock. Havelund^[16] proposed GoodLock based on lock trees. Agarwal et al.^[17] improved GoodLock to detect deadlocks caused by the arbitrary number of threads. Joshi et al.^[18] proposed a cyclic lock dependence chain to extend lock graphs with both thread ID and lock set information, which can eliminate false positives caused by cycles within one thread or by cycles guarded by a gate lock. Cai and Chan^[19] proposed Magiclock to improve the deadlock detection performance by eliminating removable lock dependence in a cyclic lock dependence chain. However, these methods cannot eliminate false positives caused by cycles between code fragments that have the causality relationship due to thread start-join. To solve this problem, Bensalem and Havelund^[20] and Agarwal et al.^[21] both proposed a segmentation graph by segmenting program operations according to thread start/join statements and extended traditional lock graphs with thread ID, lock set, and segmentation information. Samak and Ramanathan^[22] used logic clocks to model causal dependence caused by thread start/join statements.

To reduce analysis overhead, Zhou et al.^[23] proposed an Undead method that prunes unnecessary information, records each thread's unique lock dependence, skips all duplicate lock dependence, and creates a monitor thread to detect the presence of cyclic lock dependence chains periodically. Cai et al.^[11] proposed AirLock to fit for in-house testing and deployment programs. It maintains a small predictive lock reachability graph and searches cycles in it, which is an exponential-time algorithm. To ensure executability of deadlock detection, Cai et al.^[12] built an executable event set through a running trace, constructed a graph based on it, built an executable path, and observed whether it generates deadlocks.

Except for lock graph-based and segmentation graph-based deadlock detection methods, researchers study deadlock replay to eliminate false positives caused by information loss in a program by using its active scheduling intervention information and conclude that a potential deadlock replayed successfully must be a real deadlock. Burckhardt et al.^[24] and Cadar et al.^[25] separately proposed similar random scheduling method for defect recurrence, which can trigger concurrent defects through a large number of programs running under different scheduling schemes. Considering

deadlocks are low probability events, DeadlockFuzzer^[18], WOLF^[22], MagicFuzzer^[26], ASN^[27], ConLock^[28], and ConLock+^[29] design a specific program scheduling scheme for each potential deadlock to increase its recurrence success probability. Among them, DeadlockFuzzer, WOLF, and MagicFuzzer propose a heuristic scheduling strategy to increase deadlock-triggering probability and block a thread when it reaches a potential deadlock point. ASN optimizes intervention timing of program scheduling by extracting constraints from deadlock candidates and necessary conditions for concurrency defect-triggering to trigger deadlocks. ConLock and ConLock+ use MagicLock to detect constraints that may lead to deadlocks and then schedule program running according to detected constraints.

However, the above methods suffer from the following problems:

(1) Their segmentation graphs only describe causal dependence between operations in the same thread or different threads because of start/join operations. However, there exists causal dependence in different threads when release and acquisition operations of a lock are coupled with thread start operations.

(2) Traditional lock graphs only record a currently-held-lock set when a lock acquisition operation is executed and eliminate false positives caused by guard-lock cycles. However, some are acquiring and releasing lock operations during the above process, which may lead to deadlock false positives.

(3) Existing deadlock replay cannot derive an intuitive scheduling schema and generate deadlock trigger paths.

To solve them, this work presents a deadlock detection and replay of multi-thread programs based on refined segmentation and lock graphs. Its main contributions are as follows:

(1) It adds statement execution order information to a lock graph to distinguish lock acquisition operations of different rounds in a cycle, based on which an order-augmented lock graph is presented.

(2) It divides segments in a segmentation graph to describe operation causal dependence by expanding lock acquisition and release information, based on which a lock-augmented segmentation graph is proposed.

(3) It proposes a deadlock detection method based on an order-augmented lock graph and a lock-augmented segmentation one, which can eliminate deadlock false

positives caused by single-thread cycles, guard-lock cycles, causal dependence caused by thread start/join operations among multi-threads, causal dependence between lock release/acquisition and thread start operations, and historical-held-locks.

Section 2 analyzes the motivation of this work with cases. Section 3 introduces a lock-augmented segmentation graph and an order-augmented lock one. Section 4 proposes a deadlock detection algorithm. Section 5 presents a deadlock replay method based on lock authorization thread sequences. Section 6 implements the proposed method in a prototype system and conducts experimental evaluation with examples. Finally, Section 7 concludes and outlooks this work.

2 Running Case and Motivation Analysis

2.1 Trace of a multi-thread program

We define a trace of a multi-thread program to mine its execution of deadlock-related concurrency primitives, which is used to construct a lock-augmented segmentation graph.

Definition 1 (Trace of a multi-thread program).

For a multi-thread program, its trace is the running trace formed by execution of deadlock-related concurrency primitives, i.e.,

$$\sigma \in \text{ConPrimitive}^* \quad (1)$$

where ConPrimitive^* is a sequence of concurrent operations, $\text{ConPrimitive} = \{\text{start}(u, v), \text{stop}(v), \text{join}(u, v), \text{acq}(u, l), \text{rel}(u, l)\}$ is a set of concurrent primitives, u and v are threads, and l is a lock.

2.2 Running cases

Traces σ_1 , σ_2 , and σ_3 in Tables 1–3 all have three threads, i.e., main , t_1 , and t_2 threads. Each event e_i in these traces corresponds to one of the defined concurrency primitives and belongs to a specific thread u . Specifically, if e_i is $\text{start}(v)$, it represents that u starts thread v ; if e_i is $\text{stop}(u)$, it represents that u terminates; if e_i is $\text{join}(v)$, it means that u synchronously waits until v terminates; if e_i is $\text{acq}(l)$, it means that u acquires lock l ; and if e_i is $\text{rel}(l)$, it means that u releases l . In these tables, the locks are represented by m , n , p , q , G , o_1 , and o_2 . Each identifier corresponds to a unique lock object in the system.

Trace σ_1 contains two real deadlocks and two false ones. However, traditional dynamic deadlock detection methods cannot distinguish this situation, which leads

Table 1 Trace σ_1 of a program with two real deadlocks and two false deadlocks. The symbol “—” indicates that the corresponding event did not occur in the thread associated with that column.

Event ID	Event e_i		
	main	t_1	t_2
1	start(t_1)	—	—
2	—	acq(G)	—
3	—	start(t_2)	—
4	—	acq(o_1)	—
5	—	acq(o_2)	—
6	—	rel(o_2)	—
7	—	rel(o_1)	—
8	—	rel(G)	—
9	—	acq(G)	—
10	—	acq(o_1)	—
11	—	acq(o_2)	—
12	—	rel(o_2)	—
13	—	rel(o_1)	—
14	—	rel(G)	—
15	—	stop(t_1)	—
16	—	—	acq(G)
17	—	—	rel(G)
18	—	—	acq(o_2)
19	—	—	acq(G)
20	—	—	rel(G)
21	—	—	acq(o_1)
22	—	—	rel(o_1)
23	—	—	rel(o_2)
24	—	—	stop(t_2)
25	join(t_1)	—	—
26	stop(main)	—	—

to deadlock false positives.

There are two cyclic waitings in trace σ_2 in Table 2. However, traditional methods cannot distinguish these situations, which leads to deadlock false positives.

Trace σ_3 in Table 3 does not have a deadlock. A $\text{park}(\cdot)$ operation will block a thread, and $\text{unpark}(\cdot)$ will unblock the blocked thread. However, existing methods cannot distinguish this situation, which leads to deadlock false positives. Note that ConPrimitive does not include $\text{park}(\cdot)$ or $\text{unpark}(\cdot)$, hence Table 3 does not provide corresponding eventIDs for these operations.

2.3 Motivation analysis

In this section, we analyze the motivation of this work. Based on segmentation graphs, an extended lock graph is proposed with mark information $\text{arcID} : < \text{seg}_1 \text{ID}$,

Table 2 Trace σ_2 of a program with a real deadlock and a false deadlock. The symbol “—” indicates that the corresponding event did not occur in the thread associated with that column.

Event ID	Event e_i		
	main	t_1	t_2
1	start(t_1)	—	—
2	start(t_2)	—	—
3	—	acq(m)	—
4	—	acq(n)	—
5	—	rel(n)	—
6	—	acq(q)	—
7	—	acq(p)	—
8	—	rel(p)	—
9	—	rel(q)	—
10	—	rel(m)	—
11	—	stop(t_1)	—
12	—	—	acq(n)
13	—	—	acq(m)
14	—	—	rel(m)
15	—	—	acq(p)
16	—	—	acq(q)
17	—	—	rel(q)
18	—	—	rel(p)
19	—	—	rel(n)
20	—	—	stop(t_2)
21	stop(main)	—	—

Table 3 Trace σ_3 of a program with a false deadlock. The symbol “—” indicates that the corresponding event did not occur in the thread associated with that column.

Event ID	Event e_i		
	main	t_1	t_2
1	start(t_1)	—	—
2	start(t_2)	—	—
3	—	acq(m)	—
4	—	acq(n)	—
5	—	rel(n)	—
6	—	unpark($\cdot$)	—
7	—	rel(m)	—
8	—	stop(t_1)	—
9	—	—	park($\cdot$)
10	—	—	acq(n)
11	—	—	acq(m)
12	—	—	rel(m)
13	stop(main)	—	—

$(t, \text{lockSet}), \text{seg}_2\text{ID} >$ for each directed arc $\langle \text{lock}_1, \text{lock}_2 \rangle$, where arcID represents the unique ID of an arc, t represents a thread to which a current lock application operation belongs, seg_1ID represents the segment number where t acquires lock lock_1 , seg_2ID represents the segment number where t acquires lock lock_2 , and lockSet represents a lock set held before t acquires lock lock_2 . The extended lock graph of traces σ_1 , σ_2 , and σ_3 are shown in Fig. 1.

In an extended lock graph, deadlocks are identified according to the following rules: (1) Two lock acquisition operations form a directed cycle; (2) two operations belong to different threads; (3) there is no directed path connection between segments where two operations are located; and (4) lock sets held during the execution of two operations do not intersect each other.

According to the above rules, the cycles formed by arcs 2 and 8, arcs 3 and 7, arcs 5 and 2, and arcs 3 and 6 in Fig. 1a are deadlocks. However, as described in Section 2.1, only two of them are real deadlocks. Similarly, the cycle formed by arcs 4 and 8 in Fig. 1b is a deadlock, but it is also a false positive. Finally, the cycle formed by arcs 1 and 2 in Fig. 1c is a deadlock,

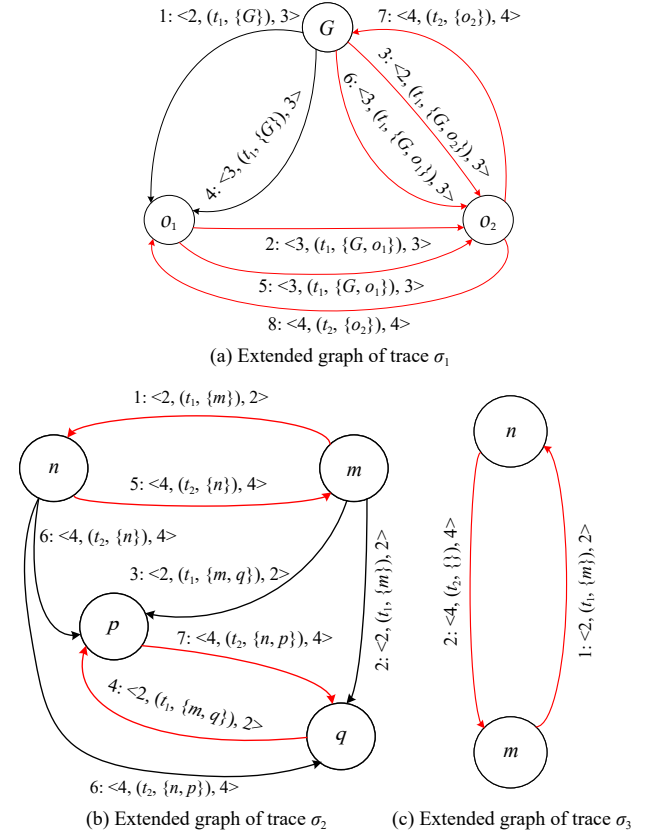


Fig. 1 Extended graphs of traces σ_1 , σ_2 , and σ_3 .

but it is a false positive.

Considering that an extended lock graph omits causal dependence and mutual exclusion information, we define causal dependence and mutual exclusion of operations to solve deadlock false positives.

Definition 2 (Causal dependence and mutual exclusion of operations). Given a running trace $\sigma = e_1, e_2, \dots, e_n$ of a multi-thread program,

(1) If events e_i and e_j belong to the same thread and $i < j$, they have intra-thread causal dependence, denoted as $e_i <_{\text{thread}} e_j$. If $e_i = \text{start}(u, v)$, $k \leq i$, $e_k.\text{thread} = u$, $m > i$, and $e_m.\text{thread} = v$, then there is inter-thread causal dependence between e_k and e_m , which is denoted as $e_k <_{\text{start}} e_m$.

(2) If $e_i = \text{join}(u, v)$, $k < i$, $e_k.\text{thread} = v$, $m \geq i$, and $e_m.\text{thread} = u$, then there is inter-thread causal dependence between e_k and e_m , which is denoted as $e_k <_{\text{join}} e_m$.

(3) Supposing that $e_i = \text{acq}(u, o)$, $e_j = \text{rel}(u, o)$, $i < j$, there is no $\text{rel}(u, o)$ between e_i and e_j , and there exists $e_m = \text{start}(u, v)$, if there exists event $e_n = \text{acq}(v, o)$ for thread v , for an integer $k \in [m, j]$, event e_l belongs to thread v , $e_k.\text{thread} = u$, and $l \geq n$, there is "lock-start" coupling causal dependence between e_k and e_l , which is denoted as $e_k <_{\text{lock_start}} e_l$.

(4) Assuming $(e_i = \text{acq}(u, o_1)) \wedge (e_j = \text{acq}(v, o_2))$, they do not have the aforementioned causal dependence. The lock set held by thread u when executing e_i is $\text{lockSet}(e_i)$. The lock set held by thread v when executing e_j is $\text{lockSet}(e_j)$. If $(\text{lockSet}(e_i) \cap \text{lockSet}(e_j)) \neq \emptyset$, there is a lock set mutual exclusion between e_i and e_j , which is denoted as $e_i \#_{\text{lockSet}} e_j$.

(5) Assuming $(e_i = \text{acq}(u, o_1)) \wedge (e_j = \text{acq}(v, o_2))$, they do not have the aforementioned causal dependence and lock set mutual exclusion. The lock set that is acquired in the process of thread u acquiring each held-lock in $\text{lockSet}(e_i)$ is $\text{lockSet_OnceHeld}(e_i)$, which is a historical-held-lock set when e_i is executed. Similarly, e_j 's historical-held-lock set is $\text{lockSet_OnceHeld}(e_j)$, we have

$$\text{L_IS} = \text{lockSet_OnceHeld}(e_i) \cap \text{lockSet}(e_j) \quad (2)$$

If e_i and e_j are concurrent to cause a deadlock, for $o \in \text{L_IS}$, event $e'_i = \text{acq}(o)$ of thread u acquiring historical-held-lock o should be executed before the last time event $e'_j = \text{acq}(o)$ of thread v acquiring held-lock o . It is said that there is a "historical holding lock-holding lock" coupling causal dependence between e'_i and e'_j , which is denoted as $e'_i <_{\text{lock_helde}} e'_j$.

From Definition 1, if intra-thread causal dependence is omitted, it may cause single-thread cycle deadlock false positives. If inter-thread causal dependence is omitted, there may exist deadlock false positives of causal cycle caused by start/join. If lockset mutual exclusion is omitted, it may lead to false positives of guard-lock cycle. If "lock-start" coupling causal dependence and "historical holding lock-holding lock" coupling causal dependence are omitted, it may cause false positives. Existing methods mainly identify false positives caused by the first three conditions based on lock graphs and various deformation models.

This work adds statement execution timing information to an extended lock graph to distinguish lock acquisition operations of different rounds in a cycle. Based on which an order-augmented lock graph is proposed. In addition, lock acquisition and release information are added to a segmentation graph to identify causal dependence caused by coupling of lock acquisition and release operations and thread start operations. Based on which a lock-augmented segmentation graph is proposed. Finally, a deadlock detection method is proposed by combining two graphs. To ensure that the detected potential deadlock is a real deadlock, we formulate a deterministic scheduling plan for potential deadlock replaying based on existing information in a lock-augmented segmentation graph.

3 Lock-Augmented Segmentation Graph and Order-Augmented Lock Graph

This section introduces the definition and construction method of a lock-augmented segmentation graph and an order-augmented lock one.

3.1 Lock-augmented segmentation graph

This section proposes a lock-augmented segmentation graph. As shown in Fig. 2, each segment in a lock-augmented segmentation graph contains information of lock acquisition and release as well as segment number. For trace σ_1 , segment 4 executes operations of acquiring lock G , releasing lock G , acquiring lock o_2 , acquiring lock o_1 , releasing lock o_1 , and releasing lock o_2 , which is represented as $(G^{(16)}, \underline{G}^{(17)}, o_2^{(18)}, G^{(19)}, \underline{G}^{(20)}, o_1^{(21)}, \underline{o}_1^{(22)}, \underline{o}_2^{(23)})$, where the number in brackets in the upper right of a lock is the operation serial number ID in a trace, a lock that is underlined indicates its release operation, and a lock that is not underlined means a lock acquisition operation. In addition,

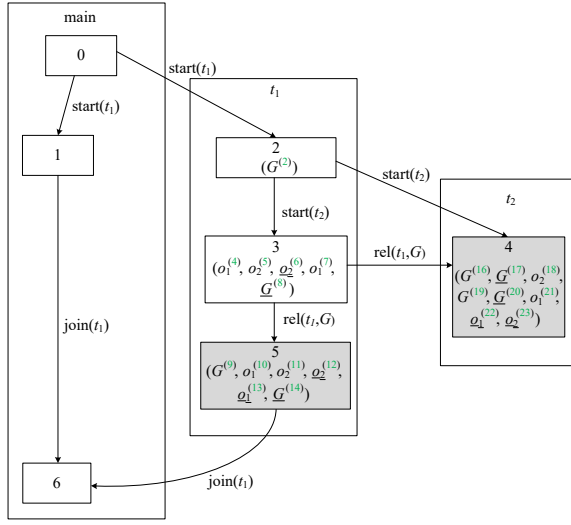
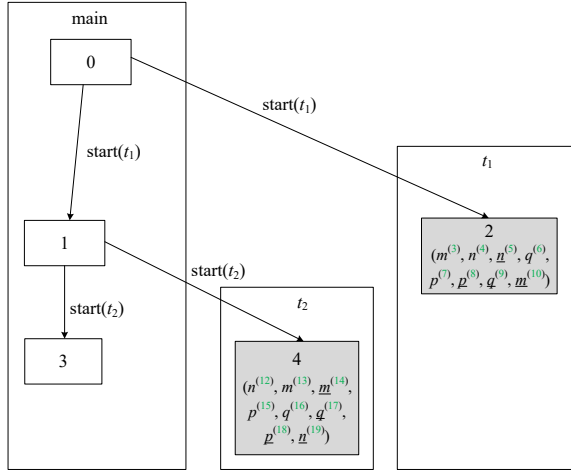
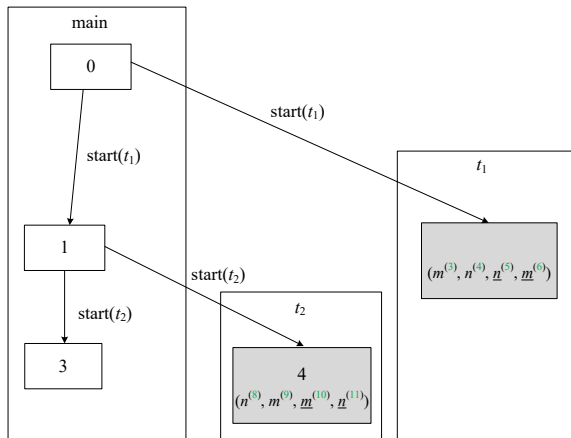
(a) Lock-augmented segmentation graph of trace σ_1 (b) Lock-augmented segmentation graph of trace σ_2 (c) Lock-augmented segmentation graph of trace σ_3

Fig. 2 Lock-augmented segmentation graphs of traces σ_1 , σ_2 , and σ_3 . The number in brackets in the upper right of a lock is the operation serial number ID in a trace, a lock that is underlined indicates its release operation, and a lock that is not underlined means a lock acquisition operation.

because thread t_1 acquires lock G in segment 2 but does not release it, “the operation of thread t_2 to acquire lock G at the beginning of segment 4” can only be executed after “operation of thread t_1 to release lock G in segment 3” ends, which is causal dependence between them. To characterize causal dependence in a lock-augmented segmentation graph, segment 3 is divided into two segments with release lock G operation as a dividing point and a directed arc is added between new segments 3 and 4 to describe their causal dependence caused by “lock-start” coupling.

For “lock-start” coupling causal dependence, its identification rules and segment/arc adding rules in a lock-augmented segmentation graph are as follows:

(1) When thread t performs a lock l release operation in segment s , we check whether its previous operation to acquire lock l is in the same segment. If not, we add a new segment for it.

(2) When thread t' performs an acquisition operation of lock l in segment s' , we reversely find the first segment node s'' that acquires lock l from this segment. If s'' does not belong to thread t' and lock l is not released in s'' , two operations are required as follows:

(i) A new segment with s' from an acquisition operation of current lock l is added and its number is s'_{new} ;

(ii) There must exist a segment s''_{desc} in descendant nodes of s'' that belongs to the same thread as s'' , which ends with the release of lock l and is a segmentation point. There is a causal dependence between s''_{desc} and s'_{new} . The operation in s'_{new} can be executed only after s''_{desc} ends and l is released. Therefore, a directed arc is added between s''_{desc} and s'_{new} .

Compared with traditional segmentation graphs, lock authorization and release information is added in a lock-augmented segmentation graph, which can be used to calculate lock information that has been held in history before each operation is executed during a deadlock detection process. Moreover, after dividing segments, causal dependence among segments caused by authorized and release operations of a lock can be described, which lays foundation for subsequent accurate deadlock detection. The definition and construction method of a lock-augmented segmentation graph are as follows, the different parts from traditional segmentation graphs are shown in bold.

Definition 3 (Lock-augmented segmentation

graph). Given trace $\sigma = e_1, e_2, \dots, e_n$ of a multi-thread program, its lock-augmented segmentation graph SegG_Lock_σ is $(\text{SegSet}_\sigma, \text{SegR}_\sigma; \varphi_s, \varphi_r)$. The composition of this graph and the definition of each component are detailed as follows:

(1) SegSet_σ is a vertex set, each vertex is a segment of a program, and its generation methods are as follows:

- (i) Segment 0 is an initial segment of a main thread;
- (ii) When thread u performs operation $\text{start}(u, v)$, a new segment is added to thread u , and an initial segment is added to thread v ;
- (iii) When thread u performs operation $\text{join}(u, v)$, a new segment is added to thread u ;
- (iv) When thread u performs operation $\text{rel}(u, l)$ in segment s , if lock l is not in segment s , a new segment is added to thread u ;
- (v) When thread v executes operation $\text{acq}(u, l)$ in segment s' , segment node s'' that is the first time to acquire lock l is reversely searched. If s'' belongs to thread $u \neq v$ and lock l is not released in s'' , s'' starts a new segment from the current acquisition operation and records its segment number as s'_{new} . The first segment with $\text{rel}(u, l)$ as tail operation in a descendant node of s'' is s''_{desc} . $(s''_{\text{desc}}, s'_{\text{new}})$ is added to relation set SegR_σ and $\text{rel}(u, l)$ is φ_r label function value corresponding to this relation;
- (vi) Each segment sets a globally unique segment number as its ID.

(2) $\text{SegR}_\sigma \subseteq \text{SegSet}_\sigma \times \text{SegSet}_\sigma$ is a relationship set, φ_r is a label function defined on SegR_σ , and its generation methods are as follows:

- (i) When thread u executes operation $\text{start}(u, v)$, the last segment before u is recorded as $\text{latest_Seg}(u)$, and the new segment added for u and v is recorded as $\text{new_Seg}(u)$ and $\text{new_Seg}(v)$. $(\text{latest_Seg}(u), \text{new_Seg}(u))$ and $(\text{latest_Seg}(u), \text{new_Seg}(v))$ are added to SegR_σ and $\text{start}(u, v)$ is their φ_r label function values;
- (ii) When thread u performs operation $\text{join}(u, v)$, the last segment before thread u is recorded as $\text{latest_Seg}(u)$, and the new segment added to u is recorded as $\text{new_Seg}(u)$. Relationship $(\text{latest_Seg}(u), \text{new_Seg}(u))$ is added to SegR_σ and $\text{join}(u, v)$ is its φ_r label function value;
- (iii) When thread v operates $\text{stop}(v)$, if thread u executed $\text{join}(u, v)$ previously, and the newly added segment of thread u due to $\text{join}(u, v)$ is supposed to be $\text{new_Seg}(u)$, and the last segment of thread v is

recorded as $\text{latest_Seg}(v)$. $(\text{latest_Seg}(v), \text{new_Seg}(u))$ is added to SegR_σ and $\text{join}(u, v)$ is its φ_r label function value;

(iv) When thread u adds a new segment due to operation $\text{rel}(u, l)$, the last segment before thread u is recorded as s' , relationship set SegR_σ is added, and $\text{rel}(u, l)$ is its φ_r label function value;

(v) When thread u adds a new segment due to lock acquisition operation $\text{acq}(u, l)$, the last segment before thread u is recorded as s' , relationship set SegR_σ is added, and $\text{acq}(u, l)$ is its φ_r label function value.

(3) φ_s is a label function defined on SegSet_σ , and its generation methods are as follows:

- (i) The φ_s label function value of an initially generated segment is empty;
- (ii) When thread u executes operation $\text{acq}(u, l)$, assuming its current segment is currentSeg and its sequence ID is k , we have

$$\varphi_s(\text{currentSeg}) := \varphi_s(\text{currentSeg}) \circ l^{(k)} \quad (3)$$

where $\circ$ represents a string splicing function;

(iii) When thread v executes operation $\text{rel}(v, l)$, assuming its segment is currentSeg and its sequence ID is k , we have

$$\varphi_s(\text{currentSeg}) := \varphi_s(\text{currentSeg}) \circ \bar{l}^{(k)} \quad (4)$$

Each segment in a lock-augmented segmentation graph corresponds to several operations of a thread. If segments s_1 and s_2 satisfy

$$(s_1, s_2) \in \text{SegR}_\sigma^+ \quad (5)$$

where superscript $+$ represents a closure operation, operations in s_2 can be executed only after all operations in s_1 are executed, and this relationship is represented as $s_1 \triangleright s_2$.

In Fig. 2a, $2 \triangleright 4$ and $3 \triangleright 4$ are valid, which means that operations in segment 4 can be executed only after operations in the second and third segments are executed. While $4 \triangleright 5$ and $5 \triangleright 4$ are not valid, which means that there is no sequential dependence on the execution order of operations between segments 4 and 5, i.e., two operation groups are concurrent.

3.2 Order-augmented lock graph

This section adds statement execution order information to an extended lock graph to distinguish lock acquisition operations of different rounds in a cycle. At the same time, operation number ID of each operation in a trace is marked. Then, an order-

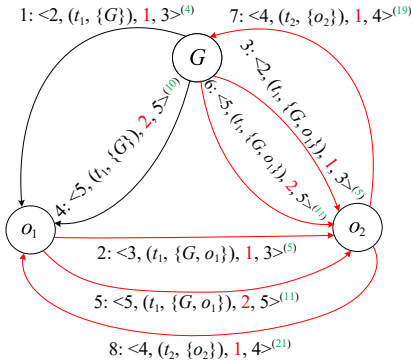
augmented lock graph is proposed.

The order-augmented lock graph of trace σ_1 in Table 1 is shown in Fig. 3a, the marking information of arc 2 is expanded from $\langle 3, (t_1, \{G, o_1\}), 3 \rangle$ in Fig. 1a to $\langle 3, (t_1, \{G, o_1\}), 1, 3 \rangle^{(5)}$, where 1 indicates that thread t_1 's operation is the first time being applied for lock o_2 , and the appended superscript digit 5 is the serial number ID of this operation on an execution trace.

An order-augmented lock graph can ensure that multiple executions of the same lock authorization statement in a cycle can be distinguished and position of each arc in it can be determined according to the serial number ID of an operation corresponding to each directed arc in a trace, which provides convenience for subsequent deadlock detection. Its definitions are as follows:

Definition 4 (Order-augmented lock graph).

Given trace $\sigma = e_1, e_2, \dots, e_n$ of a multi-thread program, its order-augmented lock graph $\text{LockG_Order}_\sigma$ is (V_σ, R_σ) . Each component in this graph is defined as follows:



(a) Order-augmented lock graph of trace σ_1

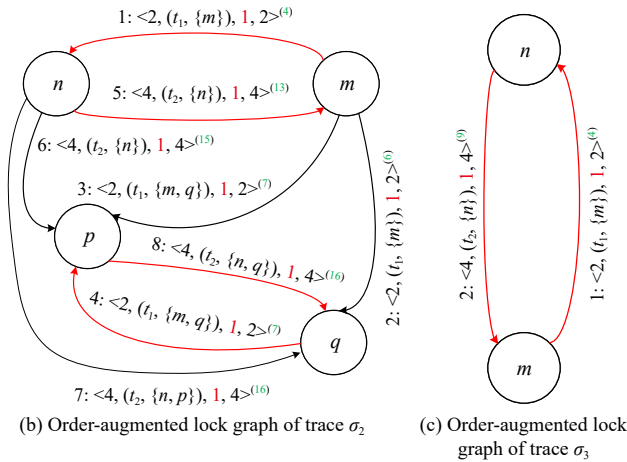


Fig. 3 Order-augmented lock graphs of traces σ_1 , σ_2 , and σ_3 .

(1) V_σ is a vertex set, and each vertex is a lock appeared in σ ;

(2) $R_\sigma \subseteq V_\sigma \times \Gamma \times V_\sigma$ is a relationship set, where Γ is a label set of directed arcs;

(3) When thread t in σ acquires lock $lock_2$ while holding lock $lock_1$, assuming the sequence ID of an operation is k , relationship $(lock_1, \gamma, lock_2) \in R_\sigma$ is added and γ is

$$\langle \text{seg}_1 \text{ID}, (t, \text{lockSet}), m, \text{seg}_2 \text{ID} \rangle^{(k)} \quad (6)$$

where $\text{seg}_1 \text{ID}$ is segment number where thread t acquired lock $lock_1$, $\text{seg}_2 \text{ID}$ is segment number where thread t acquired lock $lock_2$, lockSet is a lock set held by thread t before acquiring lock $lock_2$, and m records current operation of thread t as the number of times that thread t acquires lock $lock_2$.

(4) It is the smallest directed graph that satisfies the above conditions.

Taking traces σ_1 , σ_2 , and σ_3 as examples, their order-augmented lock graphs obtained according to Definition 4 are shown in Fig. 3. There are four directed cycles formed by arcs 2 and 8, arcs 5 and 8, 3 and 7, and arcs 6 and 7 in Fig. 3a. There are two directed cycles formed by arcs 1 and 5 and arcs 4 and 8 in Fig. 3b. There is a directed cycle formed by arcs 1 and 2 in Fig. 3c. Based on the proposed lock-augmented segmentation graph and order-augmented lock graph, we can eliminate the first three false positives. The specific methods are given in Section 4.

4 Deadlock Detection Based on A Lock-Augmented Segmentation Graph and an Order-Augmented Lock Graph

To eliminate deadlock false positives and record currently-held-lock set lockSet when an operation is executed, it is necessary to calculate its historical-held-lock sets lockSet_OnceHeld . First, the position of an operation is located in a lock-augmentation segmentation graph according to its ID. Supposing that an operation belongs to thread t , its segment number is x , and sequence number in a segment label is an index, we start from the index-1 position, access labels in a segment from right to left, and traverse labels of each segment until all locks in lockSet are acquired in a traversal process. A lock set that has appeared in the above process is lockSet_OnceHeld . The definition of The “historical holding lock-holding lock” coupling causality graph is as follows.

Definition 5 (The “historical holding lock-

holding lock” coupling causality graph). Supposing that $c = e_{k_1}, e_{k_2}, \dots, e_{k_l}$ forms a directed cycle in an order-augmented lock graph, where e_{k_i} is a lock acquisition operation. $\text{DependG_Lockc} = (V_c, R_c)$ is a “historical holding lock-holding lock” coupling causality graph where V_c is a vertex set and each vertex in it corresponds to a lock acquisition operation, $R_c \subseteq V_c \times V_c$ is an aggregate of directed edges and each directed edge is causal dependence between operations corresponding to two vertices. The generation methods of V_c and R_c are

(1) If lock o ’s operation $e_{k_i} = \text{acq}(u, o)$ and $e_{k_j} = \text{acq}(v, o)$ satisfy $o \in \text{lockSet_OnceHeld}(e_{k_i}) \cap \text{lockSet}(e_{k_j})$ and $u \neq v$, a set formed by operation e_{k_i} of thread u acquiring the historical held lock o is $\text{E_OnceHeld}(e_{k_i}, o)$ and the last operation of thread v to acquire held lock o is e'_{k_j} ;

(2) An operation with ID $(v, o^{(y)})$ is added to V_c , where y is serial number ID of operation e'_{k_j} in a program trace;

(3) For each operation $e'_{k_i} \in \text{E_OnceHeld}(e_{k_i}, o)$ of thread u acquiring historical held lock o , an operation with ID $(u, o^{(x)})$ is added to V_c , where x is serial number ID of operation e'_{k_i} in a trace. At the same time, a directed edge is added from vertex $(u, o^{(x)})$ to $(v, o^{(y)})$ to R_c ;

(4) For two vertices $(u, p^{(i)})$ and $(v, q^{(j)})$ in V_c , if $(u = v) \wedge (i < j)$, a directed edge is added from vertex $(u, p^{(i)})$ to $(v, q^{(j)})$ to R_c .

The directed edge added by (1) in Definition 5 is used to describe “historical holding lock-holding lock” coupling causal dependence. The directed edge added by (2) in Definition 5 is used to describe intra-thread causal dependence. If cycle c is a real deadlock, these dependence should be satisfied at the same time. A necessary condition for causal dependence to be satisfied at the same time is that there is no directed cycle in a “historical holding lock-holding lock” coupling causality graph. Therefore, for any directed cycle c in an order-augmented lock graph, if there is a directed cycle in its “historical holding lock-holding lock” coupling causality graph, a potential deadlock corresponding to c is a false positive.

5 Deadlock Replay

Although the proposed deadlock detection method can detect more deadlocks than traditional ones, it leads to some deadlock false positives because the trace defined

in Definition 1 ignores many primitives, such as wait, notify, suspend, `LockSupport.park`, and `LockSupport.unpark`. This section introduces a deterministic scheduling method to control operations of a program and replay potential deadlocks.

5.1 Generation of a scheduling plan

We perform deadlock replay based on scheduling of lock authorization threads. For a potential deadlock, we first calculate lock authorization thread sequences that trigger it as much as possible and then use them as scheduling schemes of deadlock replays. The related concepts in this process are defined in Definitions 6–8.

Definition 6 (Accompanying events of lock acquisition operations). Supposing σ is a trace of a multi-thread program, its order-augmented lock graph $\text{LockG_Order}_\sigma$ contains a cycle c that corresponds to a potential deadlock and $e = \text{acq}(u, o)$ is a lock acquisition operation in c , lock acquisition operations, that are executed in the process of thread u acquiring each held-lock in $\text{lockSet}(e)$, are accompanying events of e , which is denoted as $\text{accompEvents}(e)$. The union of accompanying events of lock acquisition operations in c is its accompanying event set denoted as $\text{accompEvents}(c)$.

To trigger potential deadlocks corresponding to cycle c , events in $\text{accompEvents}(c)$ cannot be executed in order of trace σ . Because we assume that σ does not trigger deadlocks, each lock acquisition operation in c can be concurrent to trigger deadlocks. Hence, events in σ are further classified.

Definition 7 (Accompanying events of deadlocks). The set of lock acquisition operations in cycle c is E_c . The set of lock acquisition operations in σ is $\sigma|_{\text{LockAcq}}$. An accompanying event set of the deadlock corresponding to c is

$$\text{DLAE}(c) = \{e | e \in \sigma|_{\text{LockAcq}} \wedge (\exists e' \in E_c : e < e')\} \quad (7)$$

The 1-order accompanying event set of the deadlock corresponding to c is

$$\begin{aligned} 1_DLAE(c) = & \text{aE}(c) \cup \{e | (e \in \sigma|_{\text{LockAcq}}) \wedge \\ & (\exists e' \in E_c : e < e') \wedge \\ & (\exists e'' \in \text{aE}(c) : e'' < e)\} \end{aligned} \quad (8)$$

The 2-order accompanying event set of the deadlock corresponding to c is a difference set $\text{DLAE}(c) - 1_DLAE(c)$ between them, which is denoted as $2_DLAE(c)$.

All events in a deadlock accompanying event set are

ancestor events of a certain lock acquisition operation in cycle c and must be executed to trigger potential deadlocks. At the same time, events in a 1-order accompanying event set are affected by accompanying events of a lock acquisition operation in cycle c . They need to meet a specific execution mode to ensure that lock acquisition operations in c are executed concurrently. Lock acquisition operations in a 2-order accompanying event set do not need to be analyzed whether lock acquisition events in cycle c can be executed concurrently. Therefore, for a potential deadlock corresponding to cycle c , lock acquisition operations in a 2-order accompanying event set are scheduled according to the thread authorization order in trace σ . For events in a 1-order accompanying event set, we construct a directed graph with each event in a 1-order accompanying event set as a node and causal dependence between them as edges. This graph is a deadlock scheduling feature relationship graph. Its strict definition is as follows. A lock authorization scheme that makes each operation in cycle c concurrent can be obtained by topologically sorting a graph.

Definition 8 (Deadlock scheduling feature relationship graph). Supposing cycle c in an order-augmented lock graph corresponds to a potential deadlock, $\text{SchdG} = (V, R)$ is a deadlock scheduling feature relationship graph corresponding to c if

$$\begin{aligned} & (v_1 <_{\text{thread}} v_2) \vee (v_1 <_{\text{start}} v_2) \vee (v_1 <_{\text{join}} v_2) \vee (v_1 <_{\text{lock_start}} v_2) \vee \\ & (v_1 <_{\text{lock_held}} v_2) \end{aligned} \quad (9)$$

where V is a vertex set, each vertex corresponds to a lock acquisition operation in $1_DLAE(c)$, and $R \subseteq V \times V$ is a directed edge set.

In summary, to trigger a potential deadlock corresponding to cycle c , the scheduling scheme of lock authorization thread during program execution is divided into two stages. In the first stage, lock acquisition operations in a 2-order accompanying event set are scheduled according to a lock authorization thread sequence in trace σ . In the second stage, lock acquisition operations in a 1-order accompanying event set are scheduled according to a lock authorization thread sequence obtained by topological sorting of the corresponding deadlock scheduling feature relationship graph. This process is shown in Fig. 4.

5.2 Deadlock replay

After obtaining a scheduling scheme of a lock, program

scheduling and deadlock replay are performed based on CalFuzzer^[29] that provides intervention functions for program execution such as lockBefore, lockAfter, and unlockAfter. Among them, a lockBefore function is automatically called before a program enters a synchronization block, a lockAfter function is automatically called after a program enters a synchronization block, and an unlockAfter function is automatically called after a program exits a synchronized block.

Based on the functions in CalFuzzer and lock scheduling schemes obtained in Section 5.1, each time a thread tries to acquire a lock, we decide whether it should be authorized according to a scheduler. Specifically, before acquiring lock o , we check whether the requesting thread should be authorized according to the first element of $\delta(o)$. There are three cases as follows:

- (1) If $\delta(o)$ is not empty and the requesting thread is not the first element of $\delta(o)$, the thread is not the expected one, we park the thread and put it into a waiting thread set associated with o ;
- (2) If $\delta(o)$ is not empty and the requesting thread is the first element of $\delta(o)$, we authorize o to the thread, update $\delta(o)$ by deleting its first element, and unpark all threads in a waiting thread set associated with o ;
- (3) If $\delta(o)$ is empty, no scheduling is required.

When running a multi-thread program, we schedule its execution with the above laws until there are no threads in RUNNABLE for TIMED_WAITING state. Finally, we check whether $\delta(o)$ is empty for lock $o \in \text{LockSet}(c)$ and CycleDetected is TRUE. If they are true, a replay process succeeds and a potential deadlock associated with c is a real deadlock. Otherwise, if $\delta(o)$ is empty for lock $o \in \text{LockSet}$ and all threads of a program terminate normally, a potential deadlock associated with c is a false deadlock. Otherwise, we cannot judge whether it is a true or false deadlock.

The proposed method is a two-stage deadlock detection method that requires running a program twice. In the first stage, CalFuzzer is used to capture a trace and an order-augmented lock graph and a lock-augmented segmentation graph are constructed based on it. Then, we analyze a potential deadlock in a program and obtain a lock authorization thread sequence corresponding to each potential deadlock and location information of a potential deadlock. In the second stage, CalFuzzer is used to perform specific intervention operations before and after lock

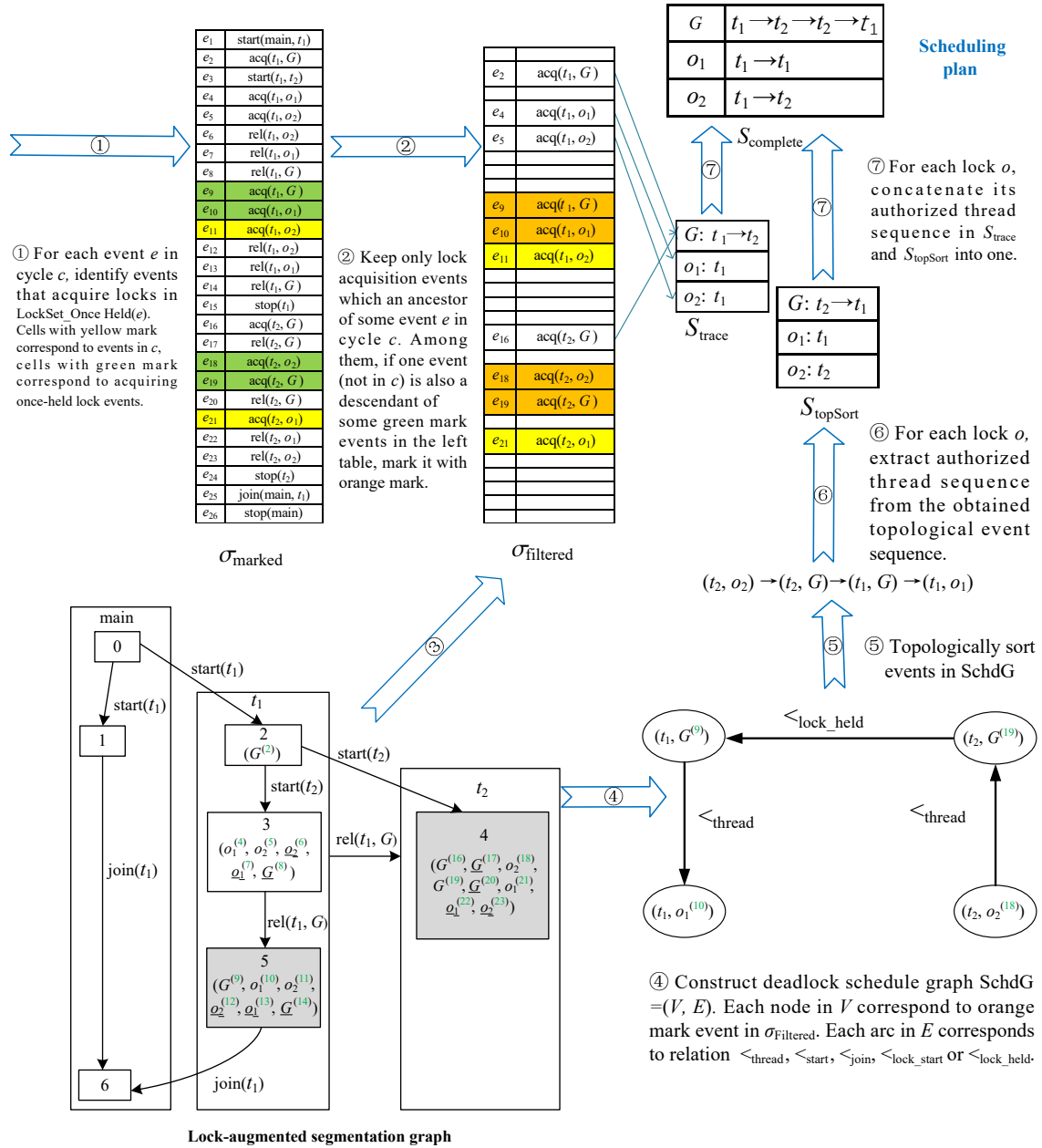


Fig. 4 Process of obtaining the scheduling scheme for a cycle.

authorization operations to realize replay of potential deadlocks. However, even in the same thread, locks or shared variables may be assigned different IDs during different runs. The prerequisite for potential deadlock replay according to the proposed method is to establish a mapping relationship between the same object with different IDs. The solutions are as follows:

(1) Because the first thread started in two runs must be the main thread, the ID correspondence between two main threads can be established. In two different program runs, the sequence of sub-threads sequentially started by a main thread is the same, so that the one-to-

one mapping of IDs can be established between them. Similarly, according to a thread sequence started by a sub-thread, the corresponding IDs between them can be established. Finally, the relationship between thread IDs is established.

(2) The lock sequence applied by each thread in turn is consistent across runs, based on which the corresponding relationship between lock IDs can be established.

(3) The sequence of shared variables accessed by each thread is consistent, and the one-to-one mapping between shared variables' IDs can be obtained.

The proposed method can detect potential deadlocks in a trace more accurately and introduce a deterministic program scheduling scheme generation method and scheduling strategy for deadlock replay, which can verify potential deadlocks more conveniently and effectively than traditional methods.

6 Experiment and Discussion

6.1 Comparison methods

We compare the performance of the proposed method with that of lock graphs, cyclic lock dependence chains, segmentation graphs, and extended lock graphs. First, the traces in Tables 1–3 are analyzed and their deadlock detection results are shown in Table 4. In Table 4, “Yes” and “No” indicate whether a deadlock exists in the case and whether the method detects deadlocks. Except for the proposed method, other methods consider each cycle in a lock graph as a deadlock. The deadlocks corresponding to Lines 1 and 3 of Table 4 are false positives because they cannot recognize the “lock-start” coupling causal dependence. The deadlock in Line 6 of Table 4 is a false positive because it cannot deal with the “historical holding lock-holding lock” coupling causal dependence. However, they may cause some operations in a cycle that cannot be concurrent. The proposed method revises traditional lock graphs and segmentation graphs and uses lock-augmented segmentation graphs and “historical holding lock-holding lock” coupling causality graphs to identify them, which can avoid these false positives in traditional methods. The cause of deadlock false positives corresponding to Line 7 of Table 4 is that existing methods do not capture primitives such as `unpark()`/`park()`.

For false positives caused by “historical hold lock-

holding lock” coupling causal dependence, there is an example in `ConLock+`, but their deadlock detection algorithm cannot exclude it and this work can eliminate it by using deadlock replay.

The proposed method can distinguish multiple executions of the same lock authorization statement in a cycle, depict dependence of various operations in different threads according to their execution order, and record lock acquisition and release operations. Based on which a more intuitive and feasible replay scheme can be generated for deadlock replay. In this work, potential deadlocks generated in a detection phase are replayed according to a deterministic replay scheme, which further ensures deadlock detection accuracy.

6.2 Prototype system and experimental evaluation

6.2.1 Prototype system

We develop a prototype system based on Java and CalFuzzer. When users enter a multi-thread program, its order-augmented lock graph and lock-augmented segmentation graph are generated and deadlock detection and replay results are demonstrated in an interactive interface. Its architecture is shown in Fig. 5, and its interactive interface is shown in Figs. 6–9.

As shown in Fig. 5, the architecture of the proposed prototype system has three layers, i.e., user input, processing, and interface layers. The user input layer requires a user to input a multi-thread program and uses CalFuzzer to mine its trace. The processing layer receives the generated trace, constructs an order-augmented lock graph and a lock-augmented segmentation graph, identifies potential deadlocks, and replays detected potential deadlocks. The interface layer shows the results of deadlock detection and replay.

Table 4 Deadlock detection results of traces σ_1 , σ_2 , and σ_3 . “Yes” and “No” indicate whether a deadlock exists in the case and whether the method detects deadlocks.

Cyclic wait	Real deadlock	Lock graph	Deadlock detection result		Our method
			Cyclic lock dependence chain	Segmentation graph and extended lock graph	
Arcs 3 and 7 in Fig. 3a	No	Yes	Yes	Yes	No
Arcs 6 and 7 in Fig. 3a	Yes	Yes	Yes	Yes	Yes
Arcs 2 and 8 in Fig. 3a	No	Yes	Yes	Yes	No
Arcs 5 and 8 in Fig. 3a	Yes	Yes	Yes	Yes	Yes
Arcs 1 and 5 in Fig. 3b	Yes	Yes	Yes	Yes	Yes
Arcs 4 and 8 in Fig. 3b	No	Yes	Yes	Yes	No
Arcs 1 and 2 in Fig. 3c	No	Yes	Yes	Yes	No

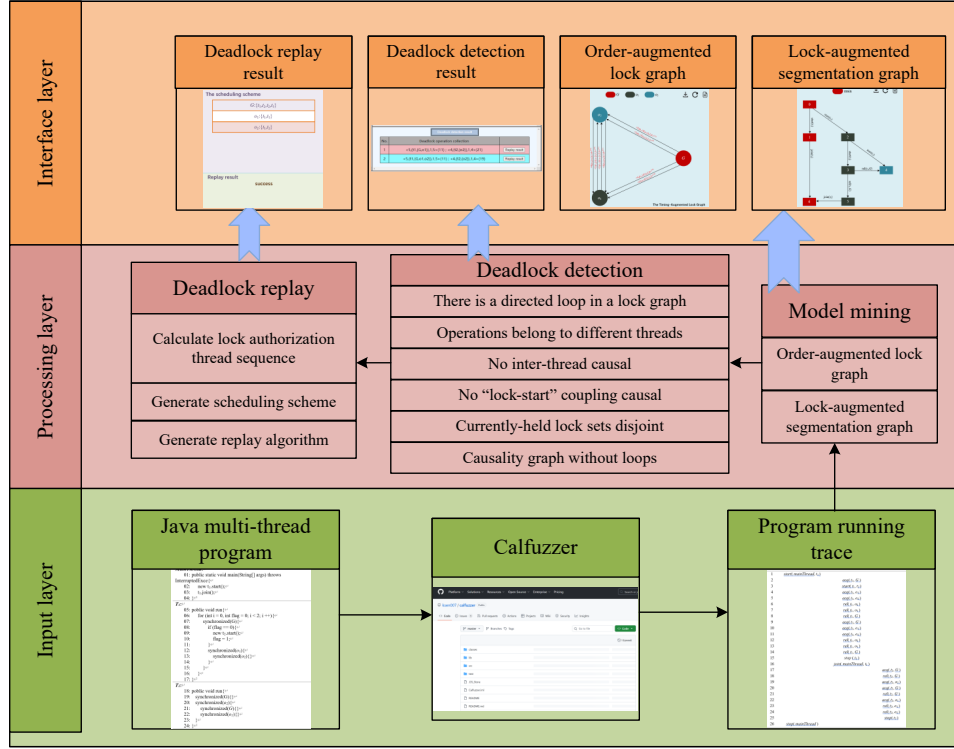


Fig. 5 Architecture of the deadlock detection and replay prototype system.

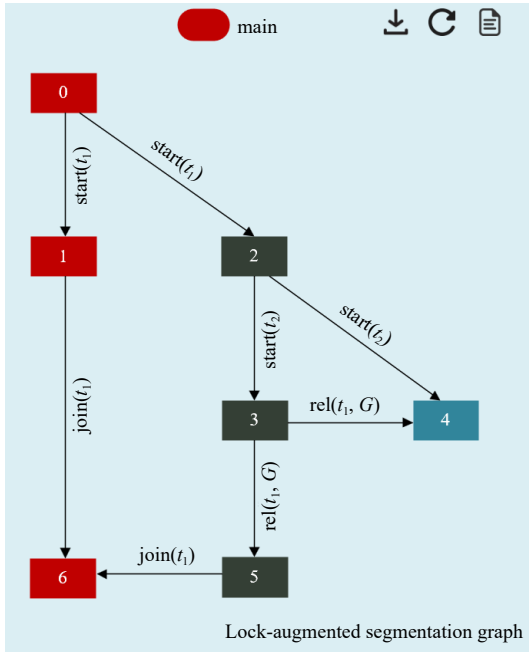


Fig. 6 The first running interface of our prototype system.

The deadlock detection result of trace σ_6 is shown in Figs. 6–9. Figure 6 shows its lock-augmented segmentation graph model generated in model mining stage, Fig. 7 shows its order-augmented lock graph model generated in model mining stage, Fig. 8 shows

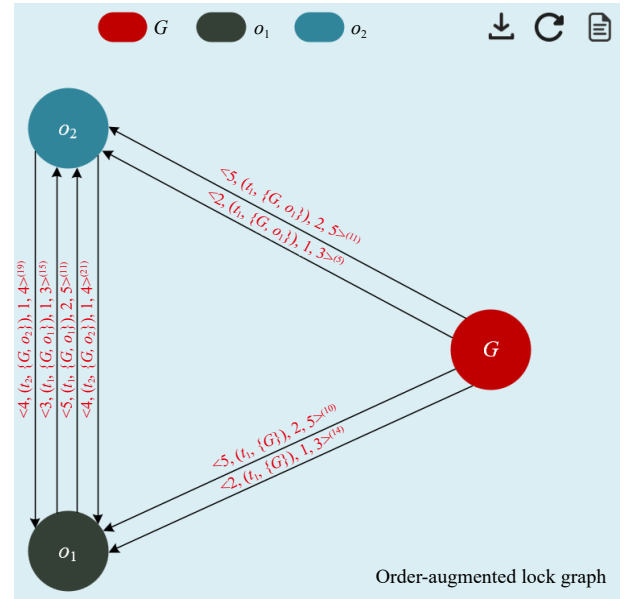


Fig. 7 The second running interface of the proposed prototype system.

its order-augmented lock graph model corresponding to potential deadlock in deadlock detection stage, and Fig. 9 shows its thread scheduling scheme and replay results in deadlock replay stage.

In Fig. 7, the left part is a lock-augmented segmentation graph that contains lock acquisition and

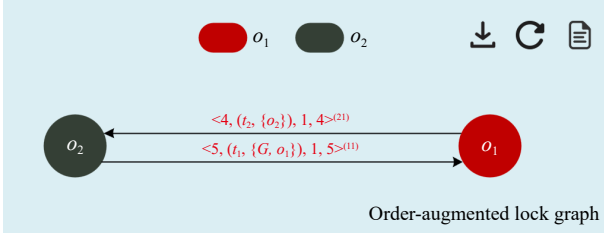


Fig.8 The third running interface of the proposed prototype system.

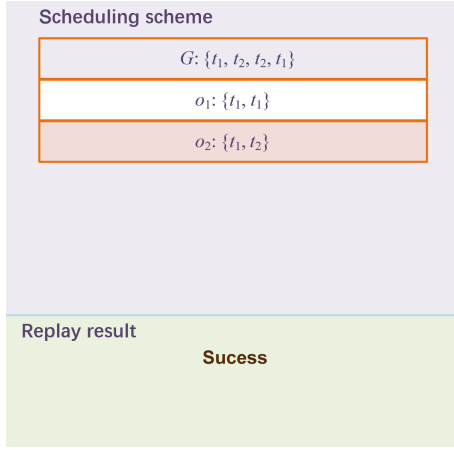


Fig.9 The fourth running interface of the proposed prototype system.

release information. The upper right is an order-augmented lock graph. The lower right is potential deadlock information.

6.2.2 Experimental evaluation

In this section, we use the Java multi-thread program

examples in Table 5 for experimental evaluation. The first eight examples are from CalFuzzer. Pingpong and cyclicDemo are from ComRaDe^[30]. ConLockDemo is from ConLock⁺^[28], which contains deadlock false positives caused by “historical holding lock-holding lock” coupling causal dependence. PetriDeadLockDemo includes deadlock false positives caused by “lock-start” coupling causal dependence. OnceHeldLockDemo is an example obtained by adding delay and operations on traces σ_1 , σ_2 , and σ_3 , which includes false positives caused by “lock-start” and “historical holding lock-holding lock” coupling causal dependence. The above examples only have deadlocks caused by two threads. To realize a more general verification, MulThreadDeadlockDemo is designed, which contains a true deadlock caused by three threads and a deadlock false positive caused by four threads.

The experimental results are shown in Table 5. PNULock^[31] is a dynamic deadlock detection method based on Petri nets. SegLock is the proposed method. We compare their performance in terms of deadlock detection accuracy, time overhead, and memory overhead. The experimental results with obvious advantages are marked in green. We conduct experiments by using Intel(R) Core(TM)i5-6300HQ CPU, 8.00 GB memory, and Ubuntu 18.10 operating system.

As shown in Table 5, iGoodlock eliminates false positives caused by single-thread cycles and guard lock cycles only, while SegLock and PNULock can

Table 5 Experimental results of deadlock detection accuracy, time overhead and memory overhead. Bold values mark the best method for the current case.

Benchmark	Potential deadlocks/false positives count			Time overhead (s)			Memory overhead (MB)	
	iGoodlock	PNULock	SegLock	iGoodlock	PNULock	SegLock	iGoodlock	SegLock
Test1a	1/0	1/0	1/0	0.198	0.291	0.157	3.695	5.219
Test 1b	1/0	1/0	1/0	0.129	0.682	0.115	3.656	4.842
Test2a	0/0	0/0	0/0	0.113	0.256	0.171	4.020	6.551
Test3	1/0	1/0	1/0	0.190	0.768	0.144	3.621	4.960
Test4	1/0	1/0	1/0	0.196	0.283	0.122	3.641	4.594
Test6	0/0	0/0	0/0	0.113	0.284	0.146	3.785	5.158
Test7	1/0	1/0	1/0	0.190	0.927	0.153	3.523	5.233
Test8	4/0	4/0	4/0	0.202	0.260	0.200	3.566	4.892
Pingpong	0/0	0/0	0/0	0.162	0.234	0.156	3.910	4.486
CyclicDemo	0/0	0/0	0/0	0.130	0.845	0.111	4.286	5.009
ConLockDemo	4/1	3/0	3/0	0.192	0.738	0.176	3.884	4.730
PetriDeadLockDemo	2/1	1/0	1/0	0.135	0.589	0.112	3.898	4.812
OnceHeldLockDemo	4/2	2/0	2/0	2.158	2.512	2.118	3.904	4.956
MulThreadDeadlockDemo	2/1	1/0	1/0	0.726	1.354	0.235	3.816	5.026

eliminate false positives caused by the inter-thread, “lock-start”, and “historical holding lock-holding lock” coupling causal dependence.

As shown in Table 5, SegLock is significantly better than PNULock in terms of time overhead because the latter’s Petri net models contain more information and it suffers from state explosion problems. When there are potential deadlocks in a program, SegLock’s time overhead performance is lower than iGoodlock’s, because the former calculates cycles in an order-augmented lock graph based on a graph matrix and its time complexity is polynomial, while the latter completes cycle detection based on graph traversal and its time complexity is exponential. When there is no potential deadlock in a program, SegLock’s time overhead performance is slightly higher than iGoodlock’s, because the former’s lock-augmented segmentation graph and order-augmented lock graph have more information and it needs to construct a “historical holding lock-holding lock” coupling causality graph and judges whether it contains cycles.

As shown in Table 5, because PNULock suffers from state explosion problems, its memory overhead is higher than SegLock and iGoodlock. SegLock’s memory overhead is higher than iGoodlock’s, because SegLock’s lock-augmented segmentation graph and order-augmented lock graph have more information and it needs to construct a “historical holding lock-holding lock” coupling causality graph.

The experimental results of deadlock replay are

shown in Table 6, where “Yes” and “No” indicate whether deadlock replay succeeds or fails, and bold values mark the best method for the current case. For examples Test2a, Test6, and cyclicDemo, SegLock only needs to perform potential deadlock detection because they do not have deadlocks. Because the SegLock and CalFuzzer integrated method replay detected potential deadlocks, there are no false positives in all examples. Compared with CalFuzzer, SegLock has fewer false positives in a deadlock detection stage and its replay takes less time, which is reflected in ConLockDemo, PetriDeadLockDemo, and OnceHeldLockDemo. Because DeadlockFuzzer has a thrashing problem in a replay stage, it needs to be run multiple times to successfully replay a potential deadlock of a program. For Test 1a and Test 1b, they need to run at least three times to successfully replay corresponding potential deadlocks. SegLock has a deterministic program scheduling scheme for each potential deadlock, which can be accurately replayed, so it is time efficient. PNULock is also based on a deterministic program scheduling scheme for potential deadlock replay, and therefore its deadlock replay time is similar to SegLock’s.

From the experimental results in Tables 5 and 6, we can conclude that our method outperforms existing methods in terms of time overhead, memory overhead, and detection accuracy in 14 Java multi-thread program examples, which verifies its effectiveness.

Table 6 Experimental results of detection time overhead and deadlock replay performance. “Yes” and “No” indicate whether deadlock replay succeeds or fails, and bold values mark the best method for the current case.

Benchmark	Detection time overhead (s)			Replay trashing		
	iGoodlock	PNULock	SegLock	iGoodlock	PNULock	SegLock
Test1a	0.469	0.126	0.125	Yes	No	No
Test1b	0.488	0.123	0.124	Yes	No	No
Test2a	0	0	0	No	No	No
Test3	0.513	0.125	0.123	No	No	No
Test4	0.484	0.125	0.124	No	No	No
Test6	0	0	0	No	No	No
Test7	0.494	0.126	0.123	No	No	No
Test8	2.441	0.520	0.525	No	No	No
Pingpong	0	0	0	No	No	No
CyclicDemo	0	0	0	No	No	No
ConLockDemo	2.422	0.513	0.513	No	No	No
PetriDeadLockDemo	0.968	0.125	0.125	No	No	No
OnceHeldLockDemo	2.432	0.235	0.231	No	No	No
MulThreadDeadlockDemo	1.012	0.126	0.125	No	No	No

7 Summary and Future Work

This work focuses on deadlock detection in multi-thread programs. It constructs an order-augmented lock graph and a lock-augmented segmentation graph, based on which a deadlock detection method is proposed. In addition, it proposes a deterministic program scheduling scheme generation method and scheduling strategy for deadlock replay. However, this work only considers five concurrent primitives, i.e., start, stop, and join of threads as well as release and acquisition of locks. In the future, we will consider other concurrent primitives, e.g., wait, notify, and notifyAll of threads. Considering that artificial intelligence methods, e.g., deep learning and large language models, show excellent performance in vulnerability detection, we will study deep learning-based^[32, 33] and large language model-based^[34–39] deadlock detection methods. In addition, we will study automatic program repair^[40] based on detection results.

Acknowledgment

This work was supported by the National Key R&D Program of China (No. 2022ZD0119501), the National Natural Science Foundation of China (Nos. 52374221 and 72404177), the Science and Technology Development Fund of Shandong Province of China (Nos. ZR2022MF288 and ZR2023MF097), the Taishan Scholar Program of Shandong Province (No. ts20190936), the Natural Science Foundation of Shandong Province of China (No. ZR2024QF107), the Major Basic Research Project of the Shandong Provincial Natural Science Foundation (No. ZR2024ZD22), and the Horizontal Research Projects Commissioned by Enterprises and Institutions (No. HX20220445).

Declaration of competing interest

The authors have no competing interests to declare that are relevant to the content of this article.

References

- [1] R. Kouatly and T. A. Khan, Performance of text-independent automatic speaker recognition on a multicore system, *Tsinghua Science and Technology*, vol. 29, no. 2, pp. 447–456, 2024.
- [2] H. Xiao, Y. Lu, J. Huang, and W. Xue, An MPI+OpenACC-based PRM scalar advection scheme in the GRAPES model over a cluster with multiple CPUs and GPUs, *Tsinghua Science and Technology*, vol. 27, no. 1, pp. 164–173, 2022.
- [3] E. Yıldız, L. Chen, and K. S. Yildirim, Immortal threads: Multithreaded event-driven intermittent computing on Ultra-Low-Power microcontrollers, in *Proc. 16th USENIX Symp. Operating Systems Design and Implementation*, Carlsbad, CA, USA, 2022, pp. 339–355.
- [4] H. Zhu, Z. Gu, H. Zhao, K. Chen, C. T. Li, and L. He, Developing a pattern discovery method in time series data and its GPU acceleration, *Big Data Mining and Analytics*, vol. 1, no. 4, pp. 266–283, 2018.
- [5] J. Zhang, C. Zhang, J. Xuan, Y. Xiong, Q. Wang, B. Liang, L. Li, W. Dou, Z. Chen, L. Chen, et al., Recent progress in program analysis, (in Chinese), *J. Softw.*, vol. 30, no. 1, pp. 80–109, 2019.
- [6] D. Wang, F. He, Y. Deng, C. Su, M. Gu, and J. Sun, Deadlock detection in FPGA design: A practical approach, *Tsinghua Science and Technology*, vol. 20, no. 2, pp. 212–218, 2015.
- [7] S. Lu, S. Park, E. Seo, and Y. Zhou, Learning from mistakes: A comprehensive study on real world concurrency bug characteristics, in *Proc. 13th Int. Conf. Architectural Support for Programming Languages and Operating Systems*, Seattle, WA, USA, 2008, pp. 329–339.
- [8] X. Su, Z. Yu, T. Wang, and P. Ma, A survey on exposing, detecting and avoiding concurrency bugs, (in Chinese), *Chin. J. Comput.*, vol. 38, no. 11, pp. 2215–2233, 2015.
- [9] J. Sun, Y. Liu, J. S. Dong, Y. Liu, L. Shi, and É. André, Modeling and verifying hierarchical real-time systems using stateful timed CSP, *ACM Trans. Softw. Eng. Methodol.*, vol. 22, no. 1, p. 3, 2013.
- [10] Y. Cai, C. Ye, Q. Shi, and C. Zhang, Peahen: Fast and precise static deadlock detection via context reduction, in *Proc. 30th ACM Joint European Software Engineering Conf. and Symp. the Foundations of Software Engineering*, Singapore, 2022, pp. 784–796.
- [11] Y. Cai, R. Meng, and J. Palsberg, Low-overhead deadlock prediction, in *Proc. ACM/IEEE 42nd Int. Conf. Software Engineering*, Seoul, Republic of Korea, 2020, pp. 1298–1309.
- [12] Y. Cai, H. Yun, J. Wang, L. Qiao, and J. Palsberg, Sound and efficient concurrency bug prediction, in *Proc. 29th ACM Joint Meeting on European Software Engineering Conf. and Symp. the Foundations of Software Engineering*, Athens, Greece, 2021, pp. 255–267.
- [13] J. Zhou, H. Yang, J. Lange, and T. Liu, Deadlock prediction via generalized dependency, in *Proc. 31st ACM SIGSOFT Int. Symp. Software Testing and Analysis*, Virtual Event, 2022, pp. 455–466.
- [14] H. C. Tunç, U. Mathur, A. Pavlogiannis, and M. Viswanathan, Sound dynamic deadlock prediction in linear time, *Proc. ACM Program. Lang.*, vol. 7, no. PLDI, pp. 1733–1758, 2023.
- [15] J. J. Harrow Jr., Runtime checking of multithreaded applications with visual threads, in *Proc. 7th Int. SPIN Workshop on Model Checking and Software Verification*, Stanford, CA, USA, 2000, pp. 331–342.

- [16] K. Havelund, Using runtime analysis to guide model checking of Java programs, in *Proc. 7th Int. SPIN Workshop on Model Checking and Software Verification*, Stanford, CA, USA, 2000, pp. 245–264.
- [17] R. Agarwal, L. Wang, and S. D. Stoller, Detecting potential deadlocks with static analysis and run-time monitoring, in *Proc. 1st Int. Haifa Verification Conf. Hardware and Software, Verification and Testing*, Haifa, Israel, 2006, pp. 191–207.
- [18] P. Joshi, C. S. Park, K. Sen, and M. Naik, A randomized dynamic program analysis technique for detecting real deadlocks, *ACM SIGPLAN Not.*, vol. 44, no. 6, pp. 110–120, 2009.
- [19] Y. Cai and W. K. Chan, Magiclock: Scalable detection of potential deadlocks in large-scale multithreaded programs, *IEEE Trans. Softw. Eng.*, vol. 40, no. 3, pp. 266–281, 2014.
- [20] S. Bensalem and K. Havelund, Dynamic deadlock analysis of multi-threaded programs, in *Proc. 1st Int. Haifa Verification Conf. Hardware and Software, Verification and Testing*, Haifa, Israel, 2006, pp. 208–223.
- [21] R. Agarwal, S. Bensalem, E. Farchi, K. Havelund, Y. Nir-Buchbinder, S. D. Stoller, S. Ur, and L. Wang, Detection of deadlock potentials in multithreaded programs, *IBM J. Res. Dev.*, vol. 54, no. 5, p. 15, 2010.
- [22] M. Samak and M. K. Ramanathan, Trace driven dynamic deadlock detection and reproduction, *ACM SIGPLAN Not.*, vol. 49, no. 8, pp. 29–42, 2014.
- [23] J. Zhou, S. Silvestro, H. Liu, Y. Cai, and T. Liu, UNDEAD: Detecting and preventing deadlocks in production software, in *Proc. 32nd IEEE/ACM Int. Conf. Automated Software Engineering*, Urbana, IL, USA, 2017, pp. 729–740.
- [24] S. Burckhardt, P. Kothari, M. Musuvathi, and S. Nagarakatte, A randomized scheduler with probabilistic guarantees of finding bugs, *ACM SIGARCH Comput. Archit. News*, vol. 38, no. 1, pp. 167–178, 2010.
- [25] C. Cadar, D. Dunbar, and D. Engler, KLEE: Unassisted and automatic generation of high-coverage tests for complex systems programs, in *Proc. 8th USENIX Symp. Operating Systems Design and Implementation*, San Diego, CA, USA, 2008, pp. 209–224.
- [26] Y. Cai and W. K. Chan, MagicFuzzer: Scalable deadlock detection for large-scale applications, in *Proc. 34th Int. Conf. Software Engineering*, Zurich, Switzerland, 2012, pp. 606–616.
- [27] Y. Cai, C. Jia, S. Wu, K. Zhai, and W. K. Chan, ASN: A dynamic barrier-based approach to confirmation of deadlocks from warnings for large-scale multithreaded programs, *IEEE Trans. Parall. Distrib. Syst.*, vol. 26, no. 1, pp. 13–23, 2015.
- [28] Y. Cai, S. Wu, and W. K. Chan, ConLock: A constraint-based approach to dynamic checking on deadlocks in multithreaded programs, in *Proc. 36th Int. Conf. Software Engineering*, Hyderabad, India, 2014, pp. 491–502.
- [29] Y. Cai and Q. Lu, Dynamic testing for deadlocks via constraints, *IEEE Trans. Softw. Eng.*, vol. 42, no. 9, pp. 825–842, 2016.
- [30] J. Gao, X. Yang, Y. Jiang, H. Liu, W. Ying, W. Sun, and M. Gu, Managing concurrent testing of data race with ComRaDe, in *Proc. 27th ACM SIGSOFT Int. Symp. Software Testing and Analysis*, Amsterdam, The Netherlands, 2018, pp. 364–367.
- [31] Y. Yuan, Y. Li, and G. Wang, A matrix algorithm for enumerating all circuits of a graph, (in Chinese), *J. Northwestern Polytechn. Univ.*, vol. 10, no. 2, pp. 204–210, 1992.
- [32] P. Chakraborty, K. K. Arumugam, M. Alfadel, M. Nagappan, and S. McIntosh, Revisiting the performance of deep learning-based vulnerability detection on realistic datasets, *IEEE Trans. Softw. Eng.*, vol. 50, no. 8, pp. 2163–2177, 2024.
- [33] M. Fu and C. Tantithamthavorn, LineVul: A transformer-based line-level vulnerability prediction, in *Proc. 19th Int. Conf. Mining Software Repositories*, Pittsburgh, PA, USA, 2022, pp. 608–620.
- [34] X. Zhou, S. Cao, X. Sun, and D. Lo, Large language model for vulnerability detection and repair: Literature review and the road ahead, *ACM Trans. Softw. Eng. Methodol.*, vol. 34, no. 5, p. 145, 2025.
- [35] L. Chen, X. Ding, M. Emani, T. Vanderbruggen, P. H. Lin, and C. Liao, Data race detection using large language models, in *Proc. SC’23 Workshops of the Int. Conf. High Performance Computing, Network, Storage, and Analysis*, Denver, CO, USA, 2023, pp. 215–223.
- [36] Y. Shen, M. Peng, F. Zhang, and Q. Wu, Data race detection via few-shot parameter-efficient fine-tuning, *J. Syst. Softw.*, vol. 222, p. 112289, 2025.
- [37] J. Chen, F. Wang, S. Pang, M. Chen, M. Xi, T. Zhao, and J. Yin, A privacy policy text compliance reasoning framework with large language models for healthcare services, *Tsinghua Science and Technology*, vol. 30, no. 4, pp. 1831–1845, 2025.
- [38] Z. Liu, Y. Bao, S. Zeng, R. Qian, M. Deng, A. Gu, J. Li, W. Wang, W. Cai, W. Li, et al., Large language models in psychiatry: Current applications, limitations, and future scope, *Big Data Mining and Analytics*, vol. 7, no. 4, pp. 1148–1168, 2024.
- [39] Z. Zhang, J. Zhuge, and X. Zhou, VPLocator: vulnerability patch localization model based on LLM, *Tsinghua Science and Technology*, vol. 31, no. 2, pp. 809–822, 2026.
- [40] B. Li, Y. He, and H. Ma, Automatic program repair: Key problems and technologies, (in Chinese), *J. Softw.*, vol. 30, no. 2, pp. 244–265, 2019.



Faming Lu received the PhD degree in computer software and theory from Shandong University of Science and Technology, Qingdao, China, in 2013. He is currently a professor at Shandong University of Science and Technology, Qingdao, China. He has published more than 30 articles in academic journals. His research interests include Petri nets and process mining.



Xiaoyu Wang received the BS degree in software engineering from Shandong University of Science and Technology, Qingdao, China, in 2021, where he is currently pursuing the PhD degree in computer software and theory at Shandong University of Science and Technology, Qingdao, China. His research interests include program analysis, vulnerability detection, and deep learning.



Guiyuan Yuan received the BS degree in computer science and technology and the MS degree in computer application technology from Shandong University of Science and Technology, Qingdao, China, in 2015 and 2018, respectively, and the PhD degree in computer science and technology from Tongji University, Shanghai, China, in 2023. He is currently a lecturer at Shandong University of Science and Technology, Qingdao, China. He has authored over 25 technical papers in journals and conference proceedings. His research interests include Internet of vehicles and autonomous cars.



Jiajing Zheng received the MS degree in computer science and technology from Shandong University of Science and Technology, Qingdao, China, in 2022. She is currently a research and development engineer at the Intelligent Computing Products Division, Dawning Information Industry Company Limited, Qingdao, China. Her research interests include software formal methods and parallel program verification.



Yunxia Bao received the MS degree in application mathematics from Shandong University of Science and Technology, Qingdao, China, in 2005. She is currently an associate professor at College of Mathematics and System Science, Shandong University of Science and Technology, Qingdao, China. Her research interests are in the areas of statistics and Petri nets.